

Testarea folosind biblioteca Jasmine

Până acum ați învățat cum să depanați codul JavaScript manual – prin consolă, verificări cu `alert()` sau `prompt`. Acum vom face un pas mai departe și vom vedea cum pot fi scrise și rulate teste automatizate folosind biblioteca Jasmine.

Jasmine este un instrument popular care permite să scrieți teste pentru funcțiile voastre o singură dată, iar apoi să le rulați de câte ori doriți, fără intervenție manuală. În acest mod puteți verifica rapid diferite situații, inclusiv intrări neașteptate, și puteți vedea imediat dacă funcția funcționează corect.

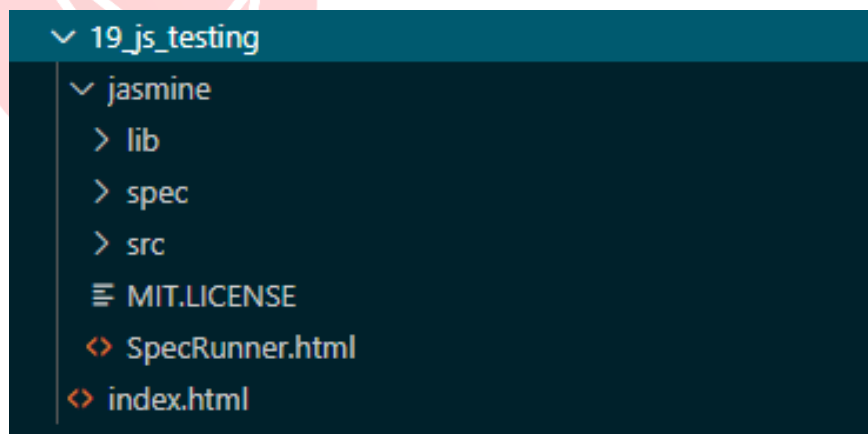
Scopul acestei lecții este să vă familiarizați cu principiul de bază de funcționare al instrumentului Jasmine, structura testelor (suite și spec), testarea funcțiilor și a cazurilor limită, precum și să obțineți o imagine de ansamblu asupra procesului de testare automatizată a codului JavaScript.

Aceasta este o introducere practică și o bază pentru explorarea ulterioară a funcționalităților mai avansate ale bibliotecii Jasmine și ale altor instrumente de testare.

Descărcarea bibliotecii Jasmine

Primul pas este descărcarea celei mai recente versiuni a acestei biblioteci:

1. Deschideți acest link în browser: <https://github.com/jasmine/jasmine/releases>
2. Descărcați versiunea standalone.
 - Căutați fișierul al cărui nume începe cu `jasmine-standalone` (de exemplu, `jasmine-standalone-6.0.0.zip`).
 - Dați clic pe fișier pentru a-l descărca.
3. Dezarhivați arhiva: după descărcare, deschideți fișierul ZIP și extrageți conținutul acestuia.
4. Plasați fișierele în proiectul vostru.
 - Se recomandă să creați un folder separat în proiect cu numele `jasmine`.
 - În interiorul aceluia folder plasați toate fișierele și folderele din arhiva dezarhivată.



Prin descărcarea bibliotecii Jasmine se obțin următoarele foldere și fișiere:

- `/lib` – folderul care conține fișierele bibliotecii Jasmine;
- `/spec` – folderul în care se plasează testele;
- `/src` – folderul în care se plasează codul JavaScript care trebuie testat;
- `SpecRunner.html` – fișierul cu care se pornește testarea.

În interiorul folderelor `spec` și `src` se află automat fișiere cu exemple de testare. În folderul `src` este plasat codul sursă demonstrativ pentru testare, în timp ce folderul `spec` se află testele. Aceste fișiere predefinite, care vin împreună cu biblioteca Jasmine, pot fi folosite pentru a înțelege modul de funcționare de bază.

Nu sunteți obligați să folosiți folderele `src` și `spec`. Puteți pune fișierele voastre oriunde doriți, însă trebuie să le includeți corect în `SpecRunner.html` pentru ca Jasmine să le poată încărca și testa.

Exemplu

Înainte să scriem primul test folosind biblioteca Jasmine, vom face o refactorizare suplimentară a codului din exemplul lecției anterioare.

Refactorizarea codului

Refactorizarea înainte de testare este importantă deoarece face codul mai clar, mai modular și mai ușor de testat. În loc să amestecăm logica, afișarea și interacțiunea cu utilizatorul, separăm funcționalitatea în funcții distincte.

De exemplu, funcția `compare(a, b)` acum doar returnează rezultatul comparației, fără mesaje alert sau prompt. Acest lucru permite testarea automată a funcției fără a fi nevoie să dăm clicuri sau să introducem manual date.

Un astfel de cod refactorizat este:

- Mai ușor de înțeles – putem vedea clar ce face funcția.
- Ușor de testat – putem scrie teste Jasmine care verifică toate cazurile.
- Modular și flexibil – ulterior putem adăuga teste noi sau modifica logica fără riscul de a strica interfața.

Prin urmare, refactorizarea înainte de testare este un pas care pregătește codul să fie testabil, curat și fiabil, lucru deosebit de important pentru începători și pentru testarea automatizată.

Refactorizarea codului – aplicare practică

1. În folderul proiectului creați un fișier `scripts.js` și mutați în el funcția `compare()`.

```
function compare(a, b) {
  let number_a = parseInt(a);
  let number_b = parseInt(b);

  if (number_a > number_b) return 1;
  else if (number_b > number_a) return -1;
  else return 0;
}
```

2. În `index.html` eliminați funcția `compare()` și faceți referire doar la fișierul `scripts.js`.

```
<script src="js/scripts.js"></script>
<script>
let number_a = prompt("Please enter first number:");
let number_b = prompt("Please enter second number:");
let result = compare(number_a, number_b);
switch (result) {
  case 1:
    alert("Number " + number_a + " is greater than " + number_b);
    break;
  case -1:
    alert("Number " + number_b + " is greater than " + number_a);
    break;
  case 0:
    alert("Numbers are equal.");
    break;
}
</script>
```

3. Creați încă un fișier JavaScript gol pentru teste, cu numele `tests.js`.

După ce am mutat funcția `compare()` într-un fișier separat (`scripts.js`) și am creat un fișier gol pentru teste (`tests.js`), următorul pas este configurarea mediului de testare Jasmine și crearea primelor teste.

Important

Toate fișierele trebuie să fie plasate în interiorul folderului proiectului, astfel încât Jasmine și browserul să le poată încărca corect. Nu puneți fișierele direct în folderul root al Jasmine!

4. Următorul pas este configurarea mediului Jasmine, pe care îl vom explica în detaliu în subcapitolul următor.

Deschiderea și editarea fișierului `SpecRunner.html`

`SpecRunner.html` este fișierul care servește la rularea testelor în browser. Deschideți acest fișier și veți vedea că el conține deja fișierele necesare ale bibliotecii Jasmine.

```
<script src="lib/jasmine-5.13.0/jasmine.js"></script>
<script src="lib/jasmine-5.13.0/jasmine-html.js"></script>
<script src="lib/jasmine-5.13.0/boot.js"></script>
```

În secțiunea `<head>` veți vedea o parte de cod cu comentarii.

```
<!-- include source files here... -->
<!-- include spec files here... -->
```

Următorul pas este să înlocuiți sau să adăugați propriile fișiere în acele locuri:

```
<!-- include source files here... -->
<script src="../../js/scripts.js"></script>
```

```
<!-- include spec files here... -->
<script src="../../tests/tests.js"></script>
```

Ce observăm aici?

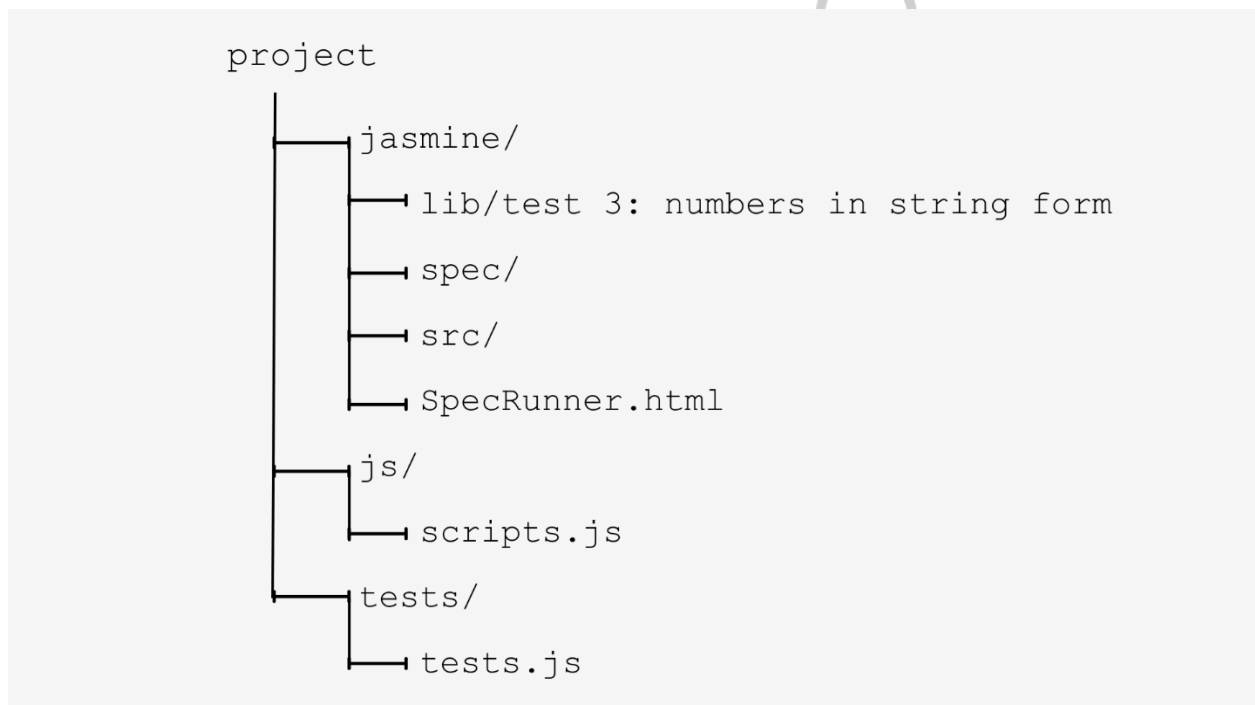
- Prima linie (`scripts.js`) permite ca Jasmine să știe unde se află funcția pe care o testăm.
- A doua linie (`tests.js`) conține testele pe care le vom scrie.

Fără aceste linii, Jasmine nu ar putea rula testele, deoarece nu ar ști unde se află codul și unde sunt testele.

Important

Source files trebuie să fie adăugate primele, iar spec files după, deoarece testele depind de codul pe care îl testează.

Structura folderului vostru arată acum astfel:



(Imaginea 02)

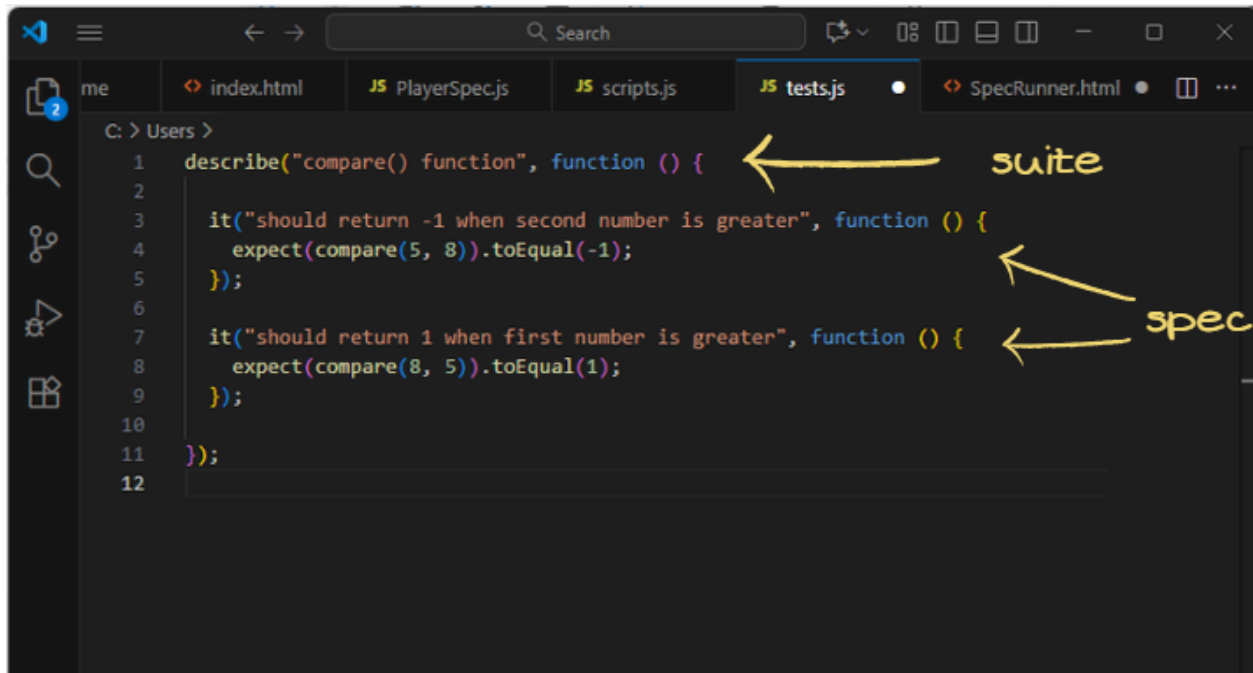
Suite și spec în Jasmine

Testele Jasmine sunt formate din două părți principale:

1. **Suite** – reprezintă un grup de teste care verifică aceeași funcționalitate.
 - Se creează cu funcția `describe()`.
 - Primul parametru este numele grupului, iar al doilea parametru este funcția care conține toate testele individuale (spec).
2. **Spec** – reprezintă un test individual în interiorul unui suite.
 - Se creează cu funcția `it()`.

- Primul parametru este descrierea testului, iar al doilea parametru este funcția care conține verificarea (`expect()`).

Suite și spec se află în fișierul cu teste, de exemplu în: `tests.js`. Acesta nu este ceva special din folderul Jasmine, ci cod JavaScript obișnuit pe care îl scrieți voi. Exemplu din `tests.js`:



```
1 describe("compare() function", function () {
2
3     it("should return -1 when second number is greater", function () {
4         expect(compare(5, 8)).toEqual(-1);
5     });
6
7     it("should return 1 when first number is greater", function () {
8         expect(compare(8, 5)).toEqual(1);
9     });
10
11 });
12
```

The image shows a code editor with a file named `tests.js` open. The code defines a Jasmine test suite. A handwritten yellow arrow points from the label "suite" to the `describe` function call on line 1. Another handwritten yellow arrow points from the label "spec" to the `it` function calls on lines 3 and 7. The code is as follows:

De ce este importantă structura Suite și Spec în Jasmine?

1. Organizare clară a testelor

Imaginați-vă că aveți o funcție și multe lucruri care trebuie verificate. Fără structura suite/spec, ați putea avea un singur test mare care verifică toate situațiile posibile. Acest lucru este greu de urmărit și derutant.

Cu Jasmine:

- `describe()` creează un grup de teste (suite) pentru o anumită funcționalitate.
- Fiecare `it()` este un test individual (spec) pentru o anumită situație.

How Jasmine "thinks"

`compare() function`

← one suite, one functionality

- └─ test 1: second number is greater
- └─ test 2: first number is greater
- └─ test 3: numbers in string form
- └─ test 4: numbers are equal

↑ multiple specs = different test cases

Acest lucru este foarte util deoarece puteți vedea clar ce se testează, iar dacă un test eșuează, știți imediat care scenariu nu funcționează.

2. Găsirea mai ușoară a erorilor

Dacă un test eșuează, Jasmine afișează clar un mesaj. Veți afla ce funcție a fost testată, care situație este problematică și ce trebuie corectat în cod.

Fără structura suite/spec, toate testele ar fi amestecate și ar fi dificil de înțeles ce nu este în regulă.

3. Testele servesc și ca documentație

Testele nu doar verifică codul, ci descriu comportamentul așteptat al funcției. De exemplu:

```
it("should return 0 when numbers are equal", function () {  
  expect(compare(5, 5)).toEqual(0);  
});
```

Știm imediat cum ar trebui să se comporte funcția și ce cazuri sunt prevăzute.

4. Standard profesional

Așa se scriu testele în proiecte reale și în echipe profesionale. Testele unitare (unit tests) sunt un standard în industrie. Când înțelegeți structura suite/spec, vă va fi mai ușor să treceți la alte instrumente: Jest, Mocha, Cypress etc.

Pe scurt, suite/spec este baza testării unitare. Această structură permite teste organizate, ușor de citit și de încredere.

Primul test

Exemplu de test simplu în fișierul `tests.js`:

```
describe("compare() function", function () {
```

```

    it("should be able to compare two numbers, when second number is
    greater", function () {
        expect(compare(5, 8)).toEqual(-1);
    });

});

```

Ce se întâmplă aici:

- `describe("compare() function", ...)` – creează un grup de teste pentru funcția `compare()`.
- `it("should be able to compare...", ...)` – creează un test care verifică un caz concret.
- `expect(compare(5, 8)).toEqual(-1)` – ne așteptăm ca funcția să returneze -1, deoarece al doilea număr este mai mare. Dacă funcția returnează altceva, testul eșuează.

Adăugarea mai multor teste

Pentru ca testarea să fie completă, trebuie să verificăm diferite situații. Adăugăm mai multe blocuri `it()` în interiorul aceluiași `describe()`:

```

describe("compare() function", function () {

    it("should return -1 when second number is greater", function () {
        expect(compare(5, 8)).toEqual(-1);
    });

    it("should return 1 when first number is greater", function () {
        expect(compare(8, 5)).toEqual(1);
    });

    it("should handle numbers in string form", function () {
        expect(compare('8', '5')).toEqual(1);
    });

    it("should handle negative numbers", function () {
        expect(compare(-8, 5)).toEqual(-1);
    });

    it("should return 0 when numbers are equal", function () {
        expect(compare(5, 5)).toEqual(0);
    });

});

```

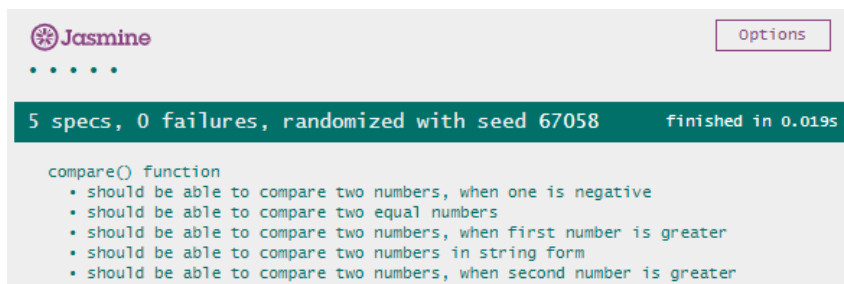
Acum, suite are mai multe spec-uri care verifică diferite situații:

1. Primul număr mai mic decât al doilea (5, 8).
2. Primul număr mai mare decât al doilea (8, 5).
3. Numere sub formă de string ('8', '5').
4. Numere negative (-8, 5).
5. Numere egale (5, 5).

Deschizând `SpecRunner.html` în browser, Jasmine va afișa automat toate testele și va semnaliza care au trecut și care nu.

După rularea `SpecRunner.html`:

- Un marcaj verde înseamnă că testul a trecut.
- Un marcaj roșu înseamnă că testul nu a trecut și afișează ce era așteptat și ce s-a obținut.



(Imaginea 04.05)

Important

Dacă nu vedeți acest rezultat:

1. Verificați consola din browser – ea va arăta dacă există erori la încărcarea fișierelor.
2. Asigurați-vă că căile către fișiere sunt corecte.
3. Fiți atenți la locația fișierului `SpecRunner.html` în raport cu fișierele `scripts.js` și `tests.js`.

Bine de știut – Sfaturi pentru definirea cazurilor de test

Când testăm o funcție cu parametri de intrare, este important să alegem cu atenție valorile pe care le transmitem.

Cele mai frecvente erori apar atunci când funcția primește valori neobișnuite sau neașteptate. Aceste valori sunt numite cazuri limită (engl. edge cases).

Edge cases

Cazurile limită sunt valori care apar rar, dar pot strica funcția. Testele automatizate ajută la identificarea imediată a acestor situații.

În exemplul nostru, funcția `compare(a, b)` așteaptă două numere. Ce se întâmplă dacă în loc de numere primim text?

```
compare('fdagdgds', 'dssdgdgsd'); // trenutno vraća 0
```

Funcția returnează în prezent 0, ceea ce înseamnă „numerele sunt egale”. Dar aceasta este o eroare, deoarece valorile transmise nu sunt numere. Așadar, funcția nu a gestionat corect tipul neașteptat de date.

Cum testăm dacă funcția reacționează corect la date invalide

În biblioteca Jasmine, putem scrie un test care verifică dacă funcția aruncă excepția corespunzătoare atunci când primește un tip greșit de date:

```
it("should throw error when values cannot be converted to number",
function () {
  expect(function() {
    compare('abc', 5);
  }).toThrowError(TypeError);
});
```

În acest cod:

- Funcția `toThrowError()` verifică dacă la apelul funcției apare un `TypeError`.
- Astfel, testul garantează că funcția reacționează corect la valori de intrare incorecte.

Corectarea funcției

Pentru a repara funcția `compare()`, adăugăm o verificare dacă vreun parametru nu este număr (`NaN`). Dacă da, aruncăm un `TypeError`:

```
function compare(a, b) {
  let number_a = parseInt(a);
  let number_b = parseInt(b);

  if (isNaN(number_a) || isNaN(number_b))
    throw new TypeError("One of the parameters cannot be converted to
number.", "scripts.js");

  if (number_a > number_b) {
    return 1;
  } else if (number_b > number_a) {
    return -1;
  } else {
    return 0;
  }
}
```

Ce observăm aici?

- `isNaN()` verifică dacă valoarea nu este un număr.
- Dacă vreun parametru nu este număr, funcția aruncă imediat excepția, iar testul verifică acest lucru.

Rulând din nou testele Jasmine, acum toate testele vor trece, inclusiv cel care verifică tipul greșit de date.

De ce este important?

- Testarea cazurilor limită descoperă erori care nu apar când folosim doar valori „normale”.
- Folosind testele Jasmine, putem identifica imediat problema și o putem corecta.
- Astfel învățăm siguranța practică a funcțiilor: funcția funcționează corect și pentru intrări neobișnuite.

Notă

Scopul acestei părți nu este să învățați biblioteca Jasmine în detaliu, ci să înțelegeți principiul de bază al testării automatizate.

Pentru mai multe informații și opțiuni avansate, consultați documentația oficială Jasmine: https://jasmine.github.io/pages/docs_home.html

Microexercițiu

Folosind biblioteca Jasmine pentru testare, testați funcția de calculare a reducerii pe baza unui cupon promoțional, pe care ați testat-o și în lecția anterioară.

Pași

- În `src/applyCoupon.js` scrieți funcția `applyCoupon`.
- În `spec/applyCoupon.spec.js` scrieți testele Jasmine.
- În `SpecRunner.html` adăugați referințele către noile fișiere `src` și `spec` (în secțiunile „include source files” și „include spec files”).
- Rulați testele deschizând `SpecRunner.html`.

